

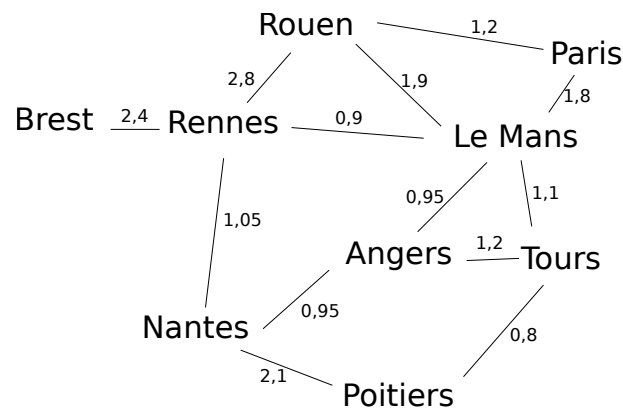
TP n°6

Programmation orientée objet en Java Clonage et Sérialisation Clonage et sérialisation

Temps de réalisation: 3h00

1 Graphes de Cartes routières

On cherche à implémenter une structure de données pour une application d'itinéraires. La fonctionnalité principale sera la recherche de chemins entre deux villes. On considérera comme exemple la carte suivante :



1.1 Structure de données

QUESTION. 1. Si vous avez déjà appris un algorithme calculant le plus court chemin entre deux nœuds d'un graphe, quelle structure de données avez-vous utilisé pour représenter les graphes dans ce calcul ?

QUESTION. 2. (*Sur papier*) Les fonctionnalités à implémenter plus loin dans ce TP sont la copie de carte et la recherche de chemin entre deux villes. En tenant compte de cette information, proposez deux architectures orientées objet différentes pour représenter une carte.

QUESTION. 3. (*Sur papier*) Comment représenter un chemin avec vos structures de données ?

QUESTION. 4. Quels sont les avantages et inconvénients de vos deux architecture dans le contexte d'un application de navigation routière ?

QUESTION. 5. En vous appuyant sur cette réflexion, sélectionnez une architecture et implémentez-la.

QUESTION. 6. Dans chacune de vos classes, redéfinissez la méthode `toString()`.

QUESTION. 7. Implémentez le graphe du schéma ci-dessus.

1.2 Copie

Lorsqu'on fait des copies d'objets, on distingue deux sortes de copies : les superficielles et les profondes. Considérons un objet *a* dont une variable d'instance *r* comporte une référence vers un objet *b*. Lors d'une copie superficielle, la variable *r* de la copie de *a* référencera *b*. Lors d'une copie profonde, on effectuera une copie de *b*, et la variable *r* de la copie de *a* référencera la copie de *b*.

QUESTION. 8. Dans la suite, on peut vouloir copier une carte pour pouvoir marquer les nœuds déjà visités par un algorithme sans modifier la carte d'origine. Dans ce but, est-il plus pertinent d'effectuer une copie superficielle ou profonde d'une carte ?

QUESTION. 9. On rappelle qu'en Java, les classes `String` et `Integer` sont immutables. En tenant compte de cette information, revoir votre réponse à la question précédente.

QUESTION. 10. Inclure dans votre architecture une méthode pour obtenir une copie d'une carte. La nature de la copie dépend de votre réponse à la question ci-dessus

Variante : lisez la documentation de l'interface `Cloneable` de Java et respectez les contraintes sur cette interface et sur la méthode `Object.clone()` pour créer vos copies.

QUESTION. 11. Une carte et sa copie sont-elles égales par `==` ? Sont-elles égales par `equals` ?

QUESTION. 12. Faites en sorte qu'une carte et sa copie soient égales par `equals`.

1.3 Recherche de chemin

QUESTION. 13. Implémentez un algorithme de recherche de chemins entre deux villes.

On n'impose pas de contraintes sur la qualité du chemin trouvé.

1.4 Sérialisation

On veut pouvoir envoyer une carte d'un serveur vers une application cliente. On va écrire les données d'un objet d'une carte dans un fichier, envoyer le fichier (*on saute cette étape dans le TP*), puis lire le fichier pour reconstituer l'objet correspondant.

L'écriture d'un objet dans un fichier s'appelle la sérialisation. La création d'un objet à partir des données d'un fichier s'appelle la désérialisation. Java fournit des mécanismes statiques et dynamiques pour assurer ces services.

QUESTION. 14. Testez le code exemple fourni pour ce TP.

QUESTION. 15. Utilisez les mécanismes de sérialisation de Java pour écrire une carte dans un fichier. *Il sera nécessaire d'exploiter l'interface `Serializable` de Java.*

QUESTION. 16. Même question pour la désérialisation.

QUESTION. 17. Afin de tester votre système, vérifiez que votre algorithme trouve le même chemin entre deux villes après sérialisation/désérialisation.

QUESTION. 18. Une carte et sa version sérialisée/désérialisée sont-elles égales par `==` ? Par `equals` ?